#8 Recursively call delete on the remaining substring, and save the result.
#9 If the key was removed, and the next node in the path was a leaf that can now be pruned, so we remove the edge to the former leaf.
#10 If, instead, the key was deleted and the next node is now a pass-through node, we can compress the path.
#11 If node `dest` is a pass-through node, it will only have 1 outgoing edge: we can retrieve it here.
#12 `dest` is a pass-through node if and only if also node has a single outgoing edge: to compress the path, we can shortcircuit the path and compress it into a single node. With this implementation, if a path has several pass-though nodes, it will be compressed one node at the time.
#13 Return, letting caller know if we deleted the key. Since for radix tries only there will only be dangling nodes, we know that pruning will certainly not be needed in this case.
#14 Match cases #2 to #4: the searched key is not in the tree, so it can't be deleted

The example in figure 6.20 shows both cases where we have to correct a node's parent: we first remove the key from a leaf, and them, once removed the node from the tree, its parent becomes a pass-through node, and hence it can be removed, allowing us to compress the path from its parent to its only child.

Listing 6.16 shows the pseudocode for this method: it uses two utility functions:

- `isPassThrough` checks if a node is a pass-through node: this only happens when a node is not a key node, it has